

一、项目概述

1.1 这是什么

GEO Explorer 是一套**品牌 AI 可见度监测与优化平台**，对标 [GEOAhead](#)。它向多个 AI 平台（DeepSeek、Kimi、豆包等）系统性地发起品牌相关的查询，监测 AI 如何描述你的品牌，检测与事实不符的"幻觉"，自动生成纠正内容，最终提升品牌在 AI 中的认知准确度。

一句话：让你的品牌在 AI 眼中是正确、完整、有竞争力的。

1.2 解决的问题

- 问题 1：你不知道 AI 怎么说你的品牌** — 用户问 AI "最好的咖啡品牌有哪些"，AI 的回答准确吗？星巴克被提到了吗？提到了但描述对吗？
- 问题 2：AI 的"幻觉"没人管** — AI 可能说星巴克是"便宜的便利店咖啡"，这是事实错误，会影响潜在客户的决策
- 问题 3：发现了问题不知道怎么修** — 传统 SEO 工具不管 AI 平台，GEO 需要新的方法论
- 问题 4：修复后没有持续监测** — 发布了纠正内容，AI 学到了吗？需要闭环验证

1.3 核心指标 (10 KPI)

KPI	中文名	说明
SOV	声量份额	品牌在 AI 回复中被提及的频率
First Recommendation Rate	首次推荐率	非品牌场景问题中优先推荐的比率
Accuracy	准确率	AI 对品牌描述与 GT（事实）的一致性
Completeness	完备性	GT 关键字段在 AI 回复中的覆盖率
Citation Rate	引用率	AI 回复中引用官方来源的比率
Scenario Recall	场景联想率	非品牌场景词下品牌被提及的比例
Semantic Stability	语义锚点稳定度	不同平台对品牌描述的一致性
Differentiation	差异化程度	AI 表述中品牌独特卖点的出现率
Cross-Platform Consistency	跨平台一致性	品牌关键声明的跨平台稳定性
Recommendation Quality	推荐理由质量	AI 推荐理由的实质性和准确度

二、系统架构

2.1 技术栈

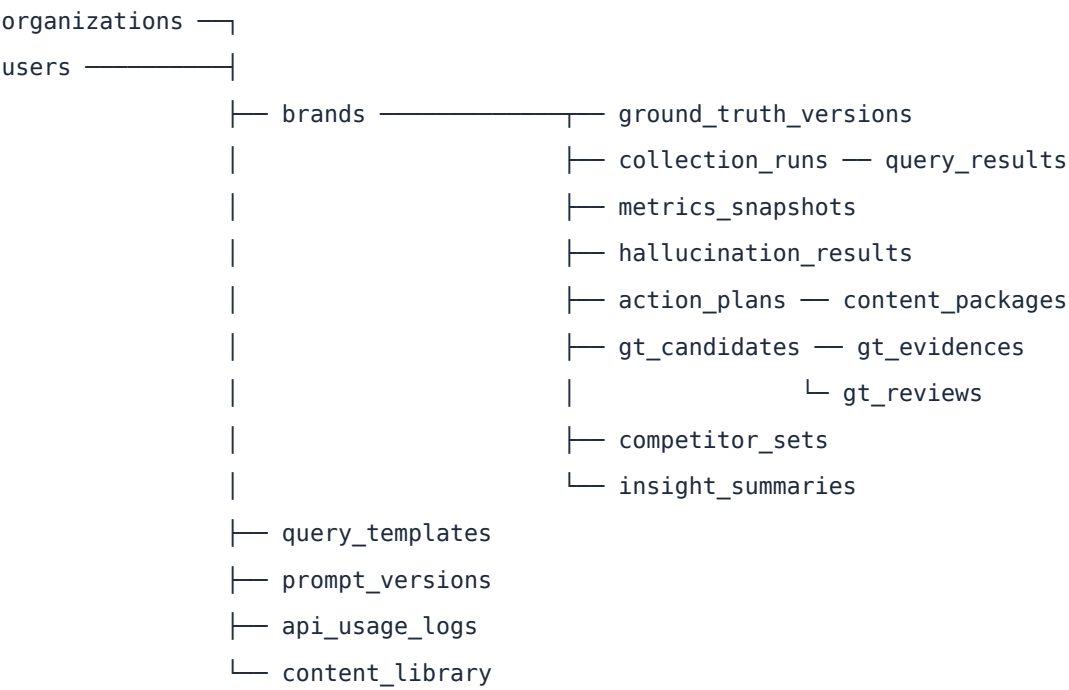
层级	技术	说明
语言	Python 3.12	全异步 (async/await)
Web 框架	FastAPI	REST API + Jinja2 + HTMX
ORM	SQLAlchemy 2.0 async	异步查询 + Alembic 迁移
数据库	PostgreSQL 16	主存储, JSONB 支持
任务队列	Celery + Redis 7	异步采集任务调度
前端	Jinja2 + HTMX + Chart.js	轻量级 SPA 体验
AI 平台	DeepSeek / Kimi / 豆包 / 文心	OpenAI-compatible API
搜索	DuckDuckGo + Google Custom Search	免费搜索 + 备用
PDF 生成	Puppeteer + marked	Chrome 渲染中文 PDF
Word 生成	python-docx	可编辑企业文档
部署	Systemd 守护进程	Redis + Celery + API 三服务

2.2 项目结构

explore geo/		# 项目根目录
— src/		
— models/	(22 文件)	# ORM 数据模型
— adapters/	(6 文件)	# AI 平台适配器
— collector/	(3 文件)	# 采集引擎 + GT 自动采集
— analyzer/	(17 文件)	# 10 KPI + 幻觉检测 + 聚合
— actions/	(6 文件)	# Action 引擎 + Content Package
— api/	(9 文件)	# REST API 端点
— search/	(3 文件)	# DuckDuckGo + AI 搜索层
— reports/	(4 文件)	# 报告生成 + PDF/DOCX 导出
— schemas/	(2 文件)	# GT 字段定义 + KPI 中文名
— tests/	(17 文件)	# 89 个测试, 1750 行
— deploy/	(3 文件)	# systemd 服务文件
— docs/superpowers/		# 设计文档 + 实现计划
— reports/		# 品牌诊断报告输出
— alembic/		# 数据库迁移

代码统计: 83 个 Python 文件, 约 4800 行源代码, 1750 行测试代码, 89 个测试 0 失败。

2.3 数据模型 (22 张表)



Phase 10 新增： GT 三层审核模型（Candidate → Evidence → Review）、Content Package 模型。

三、打造过程 — 10 个 Phase 的演进

Phase 1-2: 基础设施（8 commits）

搭建 FastAPI + PostgreSQL + Docker 基础设施，创建全部 14 个 ORM 数据模型。确立多租户架构（Organization → Brand → 业务数据），ActionPlan 状态机（pending → in_progress → completed → verified）。

Phase 3-4: 种子数据 + AI 适配器（2 commits）

22 个查询模板覆盖 5 个维度（品牌认知、竞品对比、场景推荐等），GT 字段 P0-P2 三级分级体系。5 个 AI 平台适配器统一 OpenAI-compatible 接口，支持温度控制、引用提取、搜索开关。

Phase 5-6: 采集引擎 + 分析引擎（2 commits）

异步采集引擎 with 平台级 Semaphore 限流、429 退避重试、CollectionRun 生命周期管理。5 个核心 KPI 计算器（SOV/First Rec/Accuracy/Completeness/Citation）+ 幻觉检测器。

Phase 7-8: Action 引擎 + API + Dashboard（2 commits）

ActionPlan 状态机 + Content Brief 工厂（6 类模板：FAQ/Q&A/Comparison/Tutorial/Case/Schema）。JWT 认证、CRUD API、Jinja2 Dashboard with Chart.js 可视化。

Phase 9: 采集→分析自动衔接（11 commits，62 tests）

核心突破：CollectionRun 双字段状态机 (collection_status + analysis_status)，采集完成自动触发分析，Celery 异步化 API (POST 返回 202 + task_id)。InsightSummary 模型存储分析洞察，Dashboard 展示平台健康/品牌表现/关键发现/数据可信度。

首次真实采集验证：星巴克 69/69 查询全部成功，SOV 65.2%，首次推荐率 6.7%。

Phase 10: GT 自动采集 + Action 执行 + KPI 升级（当前，89 tests）

- **GT 自动采集：**3 个 AI 平台 × 10 个 GT 问题 + DuckDuckGo 搜索 → 字段级证据聚合 → 置信度评分 → 冲突检测 → 用户审核 → Promote 为 active GT
- **KPI 升级：**从 5 个扩展到 10 个，每个含 sample_size + confidence 元数据，中文名称显示
- **幻觉检测增强：**关键词式声明提取 + n-gram 模糊分词匹配，覆盖率从 0 提升到全覆盖
- **Action Plans 自动生成：**错误声明 → P0/P1/P2 分级 → 1364 条优化任务
- **Content Package 管线：**4 主题式 LLM 生成内容 → 事实检查 → Schema.org JSON-LD → 发布检查清单
- **报告交付系统：**统一文件夹输出，3 格式 (.md/.docx/.pdf) + 独立可发布内容文件

四、完整业务链路



各环节详解

第 1 步 — GT 采集（Ground Truth Auto-Collection）

用户输入品牌名称（如"星巴克"）→ 系统向 DeepSeek/Kimi/豆包 各发送 10 个预设问题 → 同时 DuckDuckGo 搜索品牌相关信息 → 字段级证据聚合 → 每个字段生成置信度评分（high/medium/low/uncertain）→ 冲突检测 → 生成 GroundTruthCandidate（待审核）

第 2 步 — GT 审核

用户查看候选 GT → 逐字段确认（接受 / 编辑 / 删除 / 标记不确定）→ 高风险字段（official_name/category/positioning 等）强制确认 → 全部通过 → Promote 为 active GroundTruthVersion → 进入正式 GT 库

第 3 步 — Brand GEO 采集

使用 22 个标准查询模板（如"最好用的{{行业}}工具有哪些""{{品牌}}的核心功能是什么"）向 3 个 AI 平台发起系统性质询 → 采集 AI 回复 → 存入 query_results 表

第 4 步 — 分析引擎

- **10 KPI 计算：** 5 基础 + 5 扩展，每个 KPI 含样本量 + 置信度元数据
- **幻觉检测：** 从 AI 回复中提取事实性声明 → 与 active GT 进行分词模糊对比 → 标记为正确/错误/不确定
- **Action Plans 生成：** 错误声明按 P0（致命）/P1（重要）/P2（改善）分级 → 生成具体纠正任务

第 5 步 — Content Package 生成

将 Action Plans 按字段聚合为 4 大内容主题 → 使用 LLM 基于 GT 生成完整可发布内容（品牌介绍/产品FAQ/场景推荐/竞品对比）→ 事实检查 → Schema.org JSON-LD 生成 → 发布检查清单

第 6 步 — 报告交付

统一输出到一个文件夹：

- 诊断报告.md — KPI 评分 + 检测摘要
- 优化方案.md/.docx/.pdf — 三格式完整优化方案
- 0N_{{主题}}.md + _schema.json — 可直接复制到官网发布的内容资产

五、如何运行

5.1 环境要求

- Python 3.12 + venv
- PostgreSQL 16（端口 5432，用户 geo/geo，数据库 geo_explorer）
- Redis 7（端口 6379）
- Node.js 22（用于 PDF 生成）

5.2 启动命令

1. 数据库迁移

```
cd "/home/ffh/explore geo"
```

```
.venv/bin/python -m alembic upgrade head
```

2. 启动服务 (systemd 守护, 开机自启)

```
sudo systemctl start geo-redis geo-celery geo-api
```

3. 或手动启动

```
redis-server --daemonize yes
```

```
.venv/bin/celery -A src.celery_app worker --loglevel=info --concurrency=4 &
```

```
.venv/bin/python -m uvicorn src.main:app --host 0.0.0.0 --port 8000 &
```

5.3 API 使用流程

1. 创建品牌

```
curl -X POST /api/brands -d '{"name":"星巴克","industry":"餐饮"}'
```

2. 触发 GT 自动采集 (异步)

```
curl -X POST /api/brands/{{id}}/gt-collect # 返回 202 + task_id
```

3. 审核 GT + Promote

```
curl -X POST /api/gt-candidates/{{id}}/review
```

```
curl -X POST /api/gt-candidates/{{id}}/promote
```

4. 触发 GEO 诊断采集 (异步)

```
curl -X POST /api/brands/{{id}}/collections # 返回 202 + task_id
```

5. 查看 Dashboard

```
curl /api/dashboard # 10 KPI + GT 统计
```

6. 生成报告

```
curl -X POST /api/dashboard/brands/{{id}}/reports/generate
```

六、关键设计决策

6.1 为什么是 GT 三层模型

GroundTruthCandidate (候选) -> GroundTruthReview (审核记录) -> GroundTruthVersion (正式)

- Candidate 是 AI 自动采集的原始结果, 不能直接用于 KPI 计算

- 必须经过人工字段级审核（Review）后才能 Promote
- Promote 后生成 Version，旧版本自动标记为 superseded
- 这个设计确保 KPI 计算基于经过验证的事实基础

6.2 为什么字段级置信度

每个 GT 字段独立评分（high/medium/low/uncertain），而不是给整体一个分数。这样用户可以：

- 知道哪些字段是可靠的（多个来源一致）vs 需要人工判断的（来源冲突）
- 对 uncertain 字段进行针对性的人工补充

6.3 为什么不自动发布

Content Package 生成的内容经过事实检查，但**最终发布决策必须由人类做出**。原因：

- 内容可能包含品牌敏感的表述
- 发布目标（官网/CMS/第三方平台）需要人工选择
- Schema.org 结构化数据需要 SEO 验证

6.4 异步任务架构

品牌采集（66 次 AI 调用）和 GT 采集（30 次 AI 调用）都在 Celery worker 中异步执行。API 立即返回 202 + task_id，避免 HTTP 超时。这是从 Phase 9 的实际教训中得出的——同步执行时 30 个 AI 调用需要 17 分钟，HTTP 直接超时。

七、当前状态与下一步

已完成

- 89 个测试，0 失败
- 83 个 Python 源文件，约 4800 行代码
- 59 次 git 提交
- 3 份设计规范 + 3 份实现计划
- 4 个 AI 平台可正常工作（文心 API Key 待更新）
- 星巴克完整诊断：10 KPI + 1364 条 Action Plans + 4 个 Content Package 主题
- systemd 守护进程，开机自启

待完成

- **前端优化**：Dashboard 展示 10 KPI + GT 审核界面 + Content Package 管理
- **Docker 部署**：一键 docker-compose up 全套环境

- **自动发布集成:** CMS API 直连 (WordPress/Strapi/webhook)
- **文心 API Key:** 重新获取有效的百度智能云凭证

八、项目文件索引

文件/目录	说明
src/models/	22 个 ORM 数据模型
src/adapters/	5 个 AI 平台适配器
src/collector/engine.py	品牌采集引擎
src/collector/gt_collector.py	GT 自动采集编排器
src/analyzer/pipeline.py	分析管线 (KPI->幻觉->Action->Content->报告)
src/analyzer/hallucination.py	幻觉检测器
src/actions/executor.py	Content Package 生成器
src/reports/delivery.py	统一报告交付系统
src/api/dashboard.py	Dashboard + 报告 API
tests/	89 个测试
deploy/	systemd 服务配置文件
docs/superpowers/specs/	设计规范 (3 份)
docs/superpowers/plans/	实现计划 (3 份)
reports/	品牌诊断报告输出